

MemoKi: Phase 0 and Phase 1 Notes

Project: MemoKi, Episodic Memory for Large Language Models. Notes written October 2026, for future reference and for explaining the work to examiners.

1. The big picture

What the project claims: a memory system that finds relevant past conversations *faster* than normal search (flat vector search), while staying *equally accurate*. It does this by organising memory as a tree and using cheap clues (names, dates, emotion, conversation type) to skip whole branches before doing any expensive comparison.

The four gaps to be proven:

Gap	In one line
1	Nobody reports how retrieval <i>cost</i> and <i>accuracy</i> change together as memory grows
2	Nobody measures how often a conversation filed in the wrong branch becomes unreachable
3	Outdated information is handled only by LLM calls; no LLM-free rule has been tested
4	Memory systems are compared unfairly (different models, settings, copied numbers)

Where we are: Phase 0 and Phase 1 are complete. Neither phase tests the proposed method. They build the *measuring equipment* and the *test data*. The tree comes in Phase 2.

Why this order: a claim like "my method is faster at equal accuracy" is only believable if every method is measured by the same code on the same data. So the ruler is built before the thing being measured.

2. Phase 0: the foundation

2.1 What was built

Part	File	What it does	Why it exists
Settings file	configs/global.yaml	One place for embedding model, top-k (5), seeds, store sizes, split	Gap 4. No method may use private settings
Cost meter	core/counters.py	Counts cue checks, embedding comparisons, nodes visited, LLM calls, time	Gap 1. Cheap checks and expensive comparisons must be counted separately, because the whole claim is that cheap checks replace expensive ones
Data containers	core/types.py	Conversation and Result (and later Query)	Every part of the system speaks the same data

Common interface	core/retriever.py	Every method must provide <code>build()</code> and <code>_retrieve()</code> . The shared <code>retrieve()</code> creates fresh counters, starts the timer, runs the method, stops the timer	Gap 4. A method cannot time itself flatteringly or forget to reset its counters, because timing lives in shared code
First baseline	baselines/brute_force.py	Compares the query against every stored conversation	The reference every other method is judged against
Metric	eval/metrics.py	Recall@k: 1 if any correct conversation is in the top k, else 0, averaged over queries	The single accuracy number used throughout
Tests	tests/test_phase0.py	Config loads; brute force finds a known item and counts correctly; same query gives same ranking	Gap 4. Proves results are repeatable

2.2 Reasons behind the main decisions

- **Normalised vectors:** each vector is scaled to length 1, so cosine similarity becomes a plain dot product. It is faster and keeps every method on the same scale.
- **Abstract interface:** it forces fair measurement. A new method cannot skip the counters.
- **Separate counters:** an examiner can see directly how many *expensive* operations the tree avoided.
- **llm_calls counter:** it must stay at 0 for the proposed method, which is the "free and reproducible" claim in the abstract.

2.3 Result

pytest showed 3 passed. Nothing else was measured in this phase.

3. Phase 1: data and benchmark

3.1 The goal

Turn raw datasets into a reproducible benchmark: real conversations, real questions with known answers, stores of five sizes, and identical data for every method.

3.2 What was done, and why

Step	Action	Reason
1	Wrote the data pipeline (<code>pipeline/</code>) with loaders, splits, store builder, embedding cache	Everything later reads the same prepared data
2	Downloaded LoCoMo (2.8 MB) and the LongMemEval oracle file (15.4 MB)	Oracle file is small, for quick development. LoCoMo is for the Mem0/HiMem comparison later

3	Downloaded full LongMemEval (264.5 MB). The download stalled at 45 MB and was resumed with <code>curl</code>	The full file has the filler conversations needed for large stores
4	Inspected the data with <code>prepare</code>	Confirmed what exists and which store sizes are possible
5	Embedded 10,000 conversations (7 min 42 s on CPU) and cached the vectors	Embedding is slow, so do it once. Every method then uses <i>identical</i> vectors (Gap 4)
6	Ran brute force at all five sizes	First real measurement and the accuracy target for the tree
7	Tested 512 vs 256 token length	Check whether truncation was hurting accuracy
8	Compared three embedding models	Check whether a better model would raise the ceiling

3.3 Design decisions and their reasons

Decision	Reason
Each session is kept whole (never chunked)	The synopsis argues chunking destroys the conversational thread
Validation/test split (141/329 queries), seeded and saved to disk	Thresholds are tuned on validation only and results reported on test only, so results cannot be accused of tuning on the answer sheet
One fixed set of 40 queries for every store size	Any change across sizes then comes from the size, not from different questions (Gap 1)
Queries limited to 1 or 2 evidence sessions	Keeps the smallest store (100) feasible
Nested stores (100 inside 500 inside 1,000 ...)	Scaling curves compare like with like
Seeded distractor selection	Reproducible: the same seed always builds the same store
Abstention questions skipped	They have no correct conversation, so retrieval cannot be scored on them
Embedding cache	Encode once, reuse everywhere

3.4 The data we ended up with

Item	Value
Conversations in the pool	19,195

Answerable questions	470 (50 of the 500 are abstention questions)
Question types	multi-session 121, temporal-reasoning 127, knowledge-update 72, single-session-user 64, single-session-assistant 56, single-session-preference 30
Split	141 validation / 329 test
Fixed query set	40 queries, 63 evidence sessions
Store sizes available	100, 500, 1,000, 5,000, 10,000 (all feasible)
LoCoMo	272 sessions, 1,982 questions

Note for later: the 72 knowledge-update questions are the raw material for Gap 3, and the 127 temporal-reasoning questions will exercise the time cue.

4. Findings

4.1 Brute-force baseline (seed 11, 40 queries, top-5)

Store size	Recall@5	Comparisons	Median time
100	0.875	100	0.015 ms
500	0.800	500	0.027 ms
1,000	0.600	1,000	0.082 ms
5,000	0.375	5,000	0.258 ms
10,000	0.275	10,000	0.905 ms

What it shows

1. **Cost grows in a straight line.** Ten times more memory means ten times more comparisons. This is the problem in the synopsis, now measured.
2. **Accuracy falls as memory grows** (0.875 down to 0.275). More look-alike conversations compete with the right one. This supports the abstract's claim that results get noisier as the store grows.
3. **Speed is not yet a problem for brute force.** It takes under 1 ms at 10,000. A tree written in Python may not win on milliseconds at this size, so the advantage will show first in *comparison counts*. A 50k to 100k synthetic stress test is planned to find where latency crosses over.

Is this the result we wanted? For the project, yes. If brute force had stayed accurate and fast at 10,000, there would be little for the tree to improve. The decline leaves room for the proposed method.

4.2 Embedding experiments

Experiment	Result	Conclusion
256 vs 512 tokens (40 queries, size 1,000)	0.600 vs 0.575	No gain. The gap is one query, which is noise. Keep 256
MiniLM vs bge-small vs mpnet	0.620 /	Differences are within noise (about ± 5 points).

(100 queries, size 1,000)	0.660 /	mpnet is about 4 times slower to embed. Keep
	0.640	MiniLM

Frozen choice: all-MiniLM-L6-v2, 384 dimensions, 256-token limit, used by every method.

4.3 Gap status after Phase 1

Gap	Status
1 (cost vs size)	Half done: brute-force scaling table exists. The tree's table is still to come
2 (misfiling)	Not started. Needs the tree
3 (outdated info)	Data ready (knowledge-update questions). No experiment yet
4 (fair comparison)	Foundation laid: same data, queries, model, seeds, split. Ablations and Mem0/HiMem still to come

No gap is fulfilled yet. Phase 1 made fulfilling them possible.

5. Limitations (state these honestly in the report)

- 40 queries is small.** One question is worth 2.5 points, so small differences are noise. Final experiments should use more queries and report confidence intervals.
- One vector per whole conversation.** A single fact inside a long conversation gets diluted. This is the likely reason recall is low, but it has not been proven. It is the cost of the no-chunking design.
- Text is cut at 256 tokens.** It is identical for every method, so comparisons stay fair.
- Only the seed-11 store is embedded.** Seeds 22, 33, 44, 55 pick different filler, so the rest of the pool must be embedded before the final multi-seed runs.
- Only three embedding models were tested.** A second model can be run at the end as a robustness check.
- The pool tops out at 19,195 conversations.** The 50k to 100k stress test will need generated data.

6. Questions an examiner might ask

Question	Answer
Why did accuracy drop as the store grew?	More similar conversations compete, and one vector per conversation cannot always separate them. Cue-based narrowing is meant to reduce this
Why keep the smallest model?	Two text lengths and three models were compared. Differences were within measurement noise, and the small model is about 4 times faster
Why the same 40 queries at every size?	So change across sizes is caused by size, not by different questions
How do you avoid tuning on test	The split is fixed by a seed and saved. Thresholds are tuned on

data?

validation queries only

Why count cue checks and embedding comparisons separately?

The contribution is replacing expensive comparisons with cheap checks. A combined number would hide it

Why an abstract retriever interface?

So one script and one config run every method identically, and no method can bypass the measurement

If retrieval is deterministic, why use several seeds?

Seeds change which filler conversations are in the store, and give repeated latency measurements

7. Checklist

Phase 0

- ☒ Folder structure, config, counters, interface, brute-force baseline, metric
- ☒ 3 tests passing

Phase 1

- ☒ Pipeline code, 7 more tests (10 passing in total)
- ☒ LoCoMo, LongMemEval oracle and full files downloaded and verified
- ☒ All five store sizes feasible
- ☒ Splits saved, embeddings cached (16.4 MB cache file)
- ☒ Brute-force baseline measured at all sizes
- ☒ Embedding length and model decisions made
- ☐ Git commit of all code

8. What comes next: Phase 2

1. **Cue coverage check:** how many queries contain names or dates the tree can use? It decides how hard the tree can prune and how much work the soft-scoring and fallback must do.
2. **Cue extraction:** entities (spaCy), time bucket, emotion, conversation type.
3. **Tree construction:** node structure, insertion, and a splitting rule that keeps the tree balanced.
4. **Cue-guided traversal:** beam search with soft cue scoring and a flat-search fallback.
5. **Recency rule** for Gap 3.

The target the tree must meet: Recall@5 within 2 points of the brute-force table in section 4.1, with far fewer comparisons than the store size, and zero LLM calls.

9. Glossary

Term

Meaning

Embedding / vector

A conversation turned into a list of numbers so similarity can be computed

Recall@5	Fraction of questions whose correct conversation appears in the top 5 results
Distractor	A filler conversation that is not the answer, added to make memory realistic
Evidence session	The conversation that actually contains the answer to a question
Nested stores	Smaller stores are fully contained in larger ones
Seed	A number that makes random choices repeatable
Validation / test split	Validation is for tuning settings, test is for final reporting only
Cue	A cheap clue such as a name, date, emotion or conversation type
Abstention question	A question whose correct answer is "I don't know"; it has no evidence conversation

